Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Singleton kalıbı (yaratımsal kalıpların sonuncusu)
  - Amaç: bir sınıftan yalnız ve yalnız tek bir nesne oluşabilmesini sağlamak. Örnekler: işletim sisteminde tek bir yazdırma biriktiricisi (print spooler), tek bir dosya sistemi.
  - Tek nesneyi başka bir sınıfa üye yapmak sorun çıkarır: nesneyi birden fazla sınıf kullanır, hangisinin sahibi olacağı belli değildir, sahibi henüz oluşmadan başka biri ihtiyaç duyabilir. Tartışmasız çözüm, tek nesne oluşturma sorumluluğunu sınıfın kendisine vermektir.
  - Yapı (ekrandan, 00:06:00): `- uniqueInstance : Singleton {static}`, `- state : SingletonData`, `- Singleton(...)` (private kurucu), `+ getInstance() : Singleton {static}`, `+ singletonOperation()`, `+ getterMethods()`.
  - `getInstance()` gövdesi (ekrandan, 00:06:00): `if (uniqueInstance == null) { uniqueInstance = new Singleton(...); } return uniqueInstance;`
  - Singleton'dan kalıtımla alt sınıf türetilebilir; ancak her alt sınıf yalnız kendi içinde tek olur. "A2'den bir örnek varsa başka bir alt sınıftan örnek olmasın" gibi bir kısıt bu kalıpla kodlanamaz.
- Yapısal (structural) kalıplara giriş ve Adapter kalıbı
  - Amaç: işlev bakımından uyumlu, ama metot imzaları bakımından uyumsuz bir istekçi (client) ile bir sunucu (adaptee) sınıfı konuşturmak.
  - Benzetme: ülkelerin priz standartları farklıdır (ör. İngiliz prizinde üç delik, uzun üst bacak önce girip alttaki deliklerin plastik kapağını açar). Seyahat adaptörü farklı fişleri birbirine uydurur.
  - Yapı (ekrandan, 00:22:30): `Client → Target {Request()}`; `Adapter` Target'tan türer ve `adaptee` adlı bir `Adaptee` üyesine sahiptir; `Adapter.Request()` gövdesi `adaptee->SpecificRequest()` çağrısıdır.
  - Target arayüzü isteğe bağlıdır; istekçi farklı türden adaptörlerle konuşacaksa ortak arayüz yararlı ve gerekli olur.
  - Kullanım koşulu: sınıfın istekçi tarafından kullanılmasının önündeki tek engel, mesaj adlarının (ve belki parametre sırasının) farklı olmasıdır.
  - Örnek: çizim programında soyut `Shape` sınıfı (location set/get, display, fill, setColor, silme). Daire çizmek kenar yumuşatma (anti-aliasing) gerektirdiği için zordur; elde eski bir `XXCircle` sınıfı vardır (`displayIt`, `fillIt` gibi farklı adlarla), kaynak kodu da yoktur. Çözüm: `Shape`'ten türeyen `Circle` sınıfı `XXCircle` türünden bir üyeye sahip olur ve her çağrıyı ona yönlendirir.
- Bridge kalıbı
  - Amaç: bir nesnenin arayüzünü (soyutlamasını) gerçeklemesinden ayırmak. Kalıbın diğer adı Handle'dır.
  - GoF örneği (ekrandan, 00:30:00): `Window {DrawText(), DrawRect()}` ile `WindowImp {DevDrawText(), DevDrawLine()}` iki ayrı hiyerarşi; aralarında `imp` adlı sahiplik ilişkisi (köprü). `DrawRect()` gövdesi dört kez `imp->DevDrawLine()` çağırır. Alt sınıflar: `IconWindow {DrawBorder()}`, `TransientWindow {DrawCloseBox()}` ile `XWindowImp`, `PMWindowImp`.
  - Köprü olmadan, her yeni pencere türü (ör. ikon pencere) için desteklenen her masaüstü ortamı başına ayrı alt sınıf yazmak gerekir.
  - Katılımcılar: Abstraction (temel işlevlerin arayüzü), RefinedAbstraction (bu arayüzün özelleşmeleri), Implementor (gerçeklemenin iç arayüzü), ConcreteImplementor.

  - Kullanım: soyutlama ile gerçekleme arasındaki bağın kalıcı olması istenmediğinde; hem soyutlamanın hem gerçeklemenin ayrı ayrı özelleşmesi gerektiğinde.
- Bridge örneği: iki çizim kitaplığı (DP1, DP2) ile şekiller
  - İlk çözüm (ekrandan, 00:40:00): soyut `Rectangle {+draw(), #drawLine()}`, alt sınıflar `V1Rectangle` (DP1'in `draw_a_line()` metodunu kullanır) ve `V2Rectangle` (DP2'nin `drawline()` metodunu kullanır).
  - Daire ve üçgen eklendikçe her şekil için her kitaplık başına bir alt sınıf gerekir; sınıf sayısı çarpımsal artar.
  - Bridge çözümü (ekrandan, 00:47:00): `Shape {+draw(), #drawLine(), #drawCircle()}` → `Rectangle`, `Circle`; `Drawing {+drawLine(), +drawCircle()}` → `V1Drawing` (DP1'i kullanır), `V2Drawing` (DP2'yi kullanır). `Shape`, `_dp` adlı `Drawing` üyesiyle köprüyü kurar.
  - Yeni şekil (ör. üçgen) için yalnız bir yeni `Shape` alt sınıfı yeterlidir.
- Composite kalıbı
  - Amaç: hiyerarşik parça-bütün ilişkilerini desteklemek; bir bütün ve tek tek parçaları istekçiye eş biçimli görünsün.
  - Örnekler: alt resimlerden, çizgilerden, dikdörtgenlerden oluşan resim; PowerPoint'te şekilleri gruplama.
  - Yapı (ekrandan, 01:20:00): `Component {Operation(), Add(Component), Remove(Component), GetChild(int)}`; `Leaf {Operation()}`; `Composite` (children listesi, `Operation()` gövdesi: `forall g in children g.Operation();`).
  - Ağaç terimleri: kök, ara düğüm (composite), yaprak (leaf). Yaprağın çocuğu olmaz. İstekçi bir Visitor olabilir.
  - Gerçekleme ayrıntıları: alt düğümün üst düğümü bilmesi gerekip gerekmediği, bir composite silinirken çocuklarının da silinip silinmeyeceği, koşula bağlı silme ya da askıya alma.
- Composite örneği: makine ve makine parçaları kataloğu
  - `ManufacturingItem` (id, ad, fabrika), `Machine` (parça listesi), `MachinePart` (parent id). Bir fabrikada üretilen bütün öğeleri listeleyen `Report.findItems()` metodu yazıldı ve çalıştırıldı.
  - `ReportV2`'de tekrarlanan kod `showItem()` metoduna çıkarıldı.
- Decorator kalıbı
  - Amaç: nesnelere çalışma anında (dinamik olarak) ek sorumluluk eklemek. Kalıtım bunu derleme zamanında ve kalıcı biçimde yapar.
  - Örnek: GUI'de metin alanına kaydırma çubuğu ve çerçeve eklemek. Kalıtımla her birleşim ayrı sınıf ister; çoklu kalıtım "diamond problem"e yol açar ve paneller için yine ek sınıf gerekir.
  - Çözüm (ekrandan, 02:13:20): "Sınıfları değil de, nesneleri birbirleri ile ilişkilendirelim." `VisualComponent {Draw()}` → `TextView`, `Decorator` (component üyesi; `Draw()` gövdesi `component->Draw()`) → `ScrollDecorator {Draw(), ScrollTo(), scrollPosition}`, `BorderDecorator {Draw(), DrawBorder(), borderWidth}`.
- Decorator örneği: makbuza üst bilgi ve alt bilgi eklemek
  - Önce kullanım ilişkisi ve if sınamaları, sonra enum ve switch-case, sonra iki Strategy kalıbı denendi. Birden fazla üst bilgi ve alt bilgi gerektiğinde ancak Decorator yeterli oldu.
  - `Component`, `SalesTicket`, soyut `TicketDecorator` (`myTrailer` üyesi ve `callTrailer()`), `Header1`, `Header2`, `Footer1`, `Footer2`, `Factory`. Kod Eclipse'te çalıştırıldı.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir veri ya da nesne, durum bilgisinin en büyük kısmı hangi nesnedeyse ona verilir
  - Bu soru Singleton'da net cevaplanamadığı için sorumluluk sınıfın kendisine verildi.
- Singleton ve Adapter basit ama pratik fikirlerdir
  - Hoca Singleton için ayrıca örnek yapmadı: UML şeması ve not kutusu, kalıbı doğru kodlamak için yeterli bilgiyi veriyor. İleride State ve Strategy kalıplarındaki gerçeklemeler Singleton olmaya adaydır.
- "Neyin değiştiğini bul ve onu sarmala"
  - Bridge örneğinde şekiller ve çizimler ayrı ayrı değişiyordu. Bunları tek hiyerarşide çaprazlamak bağlaşımı artırdı ve sınıf sayısını neredeyse üstel büyüttü. Her değişen şeyi kendi kalıtım hiyerarşisine koymak Bridge'e götürdü.

  - Yeni bir işlev için eklenmesi gereken alt sınıf sayısı çok artıyorsa kalıtım hiyerarşisi yanlış kurulmuş demektir.
- Adapter ile Bridge farkı
  - Adapter'da var olan iki şeyin birbirine uydurulması, Bridge'de ise soyutlama ile gerçeklemenin eşleştirilmesi söz konusudur. Bridge daha karmaşık bir kalıptır.
- Composite kalıbının eleştirisi
  - Kalıp kalıtımın özüne aykırıdır: Leaf, üst sınıftan `Add`, `Remove`, `GetChild` gibi kendisi için anlamsız metotları alır. Bunlara boş gövde ya da `return null` yazmak gerekir. Hoca bundan rahatsız olduğunu, ama daha karmaşık bir tasarıma değmeyebileceğini söyledi.
- Uzun bir switch-case yazarken kendinizi görürseniz bu bir uyarıdır
  - Daha doğru bir kalıtım ilişkisi kurup çok biçimliliği (polymorphism) kullanmak gerekir.
- Decorator'ın zayıf yönü (ekrandan, 02:20:00)
  - Yeni sorumluluk ancak eski sorumluluktan önce ve/veya sonra eklenebilir ya da eski sorumluluk tamamen iptal edilir. Eski işlemin adımlarının içine müdahale edilemez.
  - Dikkat: nesne zinciri kopabilir. Aradan bir dekoratörü çıkarmak zinciri kırar; çoğu zaman yeni bir zincir kurmak daha pratiktir.
- Aynı türden dekoratörlerin sırası önemlidir
  - Header ve footer'ın birbirine göre sırası önemli değildir, ama iki header (ya da iki footer) arasında sıra, zincire eklenme sırasına bağlıdır.

Kısa Tekrar Notları

- Singleton: static `uniqueInstance`, private kurucu, static `getInstance()`; null ise oluştur, sonra döndür.
- Adapter: istekçinin beklediği metodu sunar, çağrıyı uyarlanan nesnenin farklı adlı metoduna yönlendirir. Target arayüzü isteğe bağlıdır.
- Bridge: soyutlama hiyerarşisi + gerçekleme hiyerarşisi + aralarında sahiplik ilişkisi (köprü). Yeni şekil için tek sınıf, yeni çizim kitaplığı için tek sınıf eklenir.
- `protected` metot: istekçilere görünmez, alt sınıflara kalıtımla aktarılır. `private` metot kalıtımla aktarılmaz. UML'de # ile gösterilir.
- Composite: Component / Leaf / Composite; Composite'in işlemi bütün çocuklara yayılır. Leaf'te anlamsız metotlar boş gövdeli olur.
- Decorator: Component / ConcreteComponent / soyut Decorator / ConcreteDecorator. Dekoratör, sarmaladığı nesnenin işlemini çağırır, öncesine veya sonrasına kendi davranışını ekler.
- Java'nın stream tabanlı G/Ç kütüphanesi ve Swing, Decorator kalıbını kullanır.
- Çoklu kalıtım Java ve C#'ta desteklenmez; C++'ta belirsiz metotların alt sınıfta yeniden tanımlanması gerekir.

Detaylı Açıklamalar

- Singleton gerçeklemesi
  - Sınıfın kendi türünden static bir üyesi vardır. Static olması, ortak kullanım için gereklidir. Nesnenin durum bilgisini oluşturan diğer üye alanlar da aynı sınıftadır.
  - Kurucu `private` yapılır; böylece nesneyi yalnız sınıfın kendisi oluşturabilir. Hocaya göre private kurucuya Singleton dışında pek pratik bir örnek yoktur.
  - Nesneye `getInstance()` adlı static bir fabrika metodu (factory method) ile erişilir. Metot önce `uniqueInstance` alanının ilklendirilip ilklendirilmediğine bakar; ilklendirilmemişse private kurucuyu çağırır, sonra alanı döndürür.
- Adapter örneğinin ayrıntısı
  - Eski `XXCircle` sınıfında çizim için `displayIt`, doldurmak için `fillIt` gibi farklı adlı metotlar vardır. Parametre sırası da farklı olabilir (ör. önce yarıçap, sonra merkez koordinatları).
  - Yeni `Circle` sınıfı `Shape`'ten türer. Kurucusu `XXCircle` üyesini ilklendirir. `display()` metodunun yaptığı tek iş, `XXCircle` nesnesinin `displayIt()` metodunu çağırmaktır.
  - Slayttaki şemada `display`, `fill`, `setColor` metotları italik yazılmıştır; italik yazı soyut (abstract) metot demektir. Hoca bunun `{abstract}` notuyla gösterilmesinin daha

doğru olacağını söyledi.

- Bridge örneğinin ayrıntısı
  - DP1 kitaplığında çizgi çizme `draw_a_line(x1, y1, x2, y2)` sırasıyla, DP2 kitaplığında `drawline(x1, x2, y1, y2)` sırasıyla parametre alır.
  - İlk çözümde `drawLine` metodu `protected` yapılmıştır. İstekçi dört çizgiyle uğraşmamalı, yalnız `draw()` çağırmalıdır; bu yüzden metot public değildir. Alt sınıflarda yeniden tanımlanacağı için private da değildir.
  - Daire eklenince `Shape` soyut sınıfı ve `V1Circle`, `V2Circle` sınıfları da gerekir. Üçgen eklenince sınıf sayısı yine artar. Hoca bunu "çok üstel artış" diye niteledi.
  - Bridge çözümünde `Shape` sınıfı `_dp` adlı `Drawing` üyesini kurucuda alır. `protected drawLine(...)` ve `drawCircle(...)` metotları çağrıyı `_dp`'ye yönlendirir (ekrandan, 00:50:00). `Circle` kurucusu önce `super(dp)` çağırır, sonra `_x`, `_y`, `_r` alanlarını atar; `draw()` gövdesi `drawCircle(_x, _y, _r)` olur.
  - Üye alanların alt çizgiyle (`_`) başlaması zorunlu değildir. Kod parçasında üye alanı, parametreyi ve geçici değişkeni ayırt etmeyi kolaylaştıran, C++'tan kalma bir alışkanlıktır.
  - Kayıttan yüklenen bir üçgen birinci aileden ise bir `V1Drawing`, ikinci aileden ise bir `V2Drawing` nesnesiyle oluşturulur.
- Composite gerçekleme ayrıntıları
  - Bir yaprağın hangi composite'e ait olduğunu bilmesi gerekmiyorsa bu yönde ilişki konmaz. Bir parçaya yapılan iş bütün parçalara yayılacaksa yaprağa sahibini gösteren bir üye alan eklenir; yoksa her iş kök üzerinden yürütülmek zorundadır.
  - Bir ara composite silinirken çocukları da silinebilir ya da çocuklar üst composite'e bağlanabilir. Senaryoya göre iki ayrı silme metodu da kodlanabilir.
  - Bazı yapraklar belli bir durumdayken silme geçersiz olabilir, ya da silmek yerine geçici askıya alma istenebilir. Bunun için bileşenlerde etkin olup olmadığını tutan bir üye alan olabilir; composite yalnız etkin bileşenlere çağrı gönderir.
- Composite örneğinin kodu (ekrandan, 01:36:10, 01:40:00, 01:50:00)
  - `Machine.addPart(part)`: önce `part.setParentID(getMachineID())`, sonra parça listede varsa `false`, yoksa `parts.add(part)` sonucunu döndürür. `removePart` doğrudan `parts.remove(part)` döndürür; olmayan parça için ayrıca istisna oluşturulmaz. `getParts()` listenin iterator'ünü döndürür. `getParentID()` 0 döndürür, `setParentID` boştur.
  - `MachinePart` için `getParts()` null, `addPart` ve `removePart` false döndürür.
  - `Report.findItems(items, factory)` dizideki null öğeleri `continue` ile atlar, fabrika adı uyan öğeleri sayar ve yazdırır, sonra öğenin alt parçalarında aynı işi yapar.
  - Ana programda Bursa için arandığında çıktı (ekrandan, 01:50:00): `99 Krank şaftı KŞ-7`, `259 Flanş FTR-5`, `63 Distribütör DST-12`, "3 parça bulundu". Jeneratörün alt parçası olan flanş da bulundu; İstanbul'da üretilen jeneratör listelenmedi.
- Decorator örneğinin kodu (ekrandan, 02:40:00, 02:44:20, 02:50:00)
  - `Component` soyut sınıfında `abstract public void prtTicket();` vardır. `SalesTicket.prtTicket()` kendi sınıf adını yazar.
  - `TicketDecorator`: `private Component myTrailer;` alanı, bunu atayan kurucu ve `callTrailer()` metodu (`if (myTrailer != null) myTrailer.prtTicket();`). Null denetimi, zincirin sonunda `NullPointerException` almamak içindir.
  - `Header1.prtTicket()`: önce kendi adını yazar, sonra `callTrailer()`. `Footer1.prtTicket()`: önce `super.callTrailer()`, sonra kendi adını yazar.
  - `Factory.getComponent()`: `new SalesTicket()`, `new Footer1(...)`, `new Header1(...)` sırayla aynı değişkene atanır ya da tek satırda `return new Header1(new Footer1(new SalesTicket()));` yazılır. Çıktı her iki sırada da Header1, ticket, Footer1'dir.
  - Sınıf adını yazmak için `getClass().getName()` kullanıldı. Java'da her sınıfın ortak üst sınıfı `Object`'tir; her nesnenin `Class` türünden meta bilgisine erişilebilir. Daha fazlası için Java Reflection, BCEL (Byte Code Engineering Library) gibi araçlar vardır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.